



COS30019: Introduction to Artificial Intelligence

AI Planning

1



Planning

- The Planning problem
- Planning with **State-space** search
- Planning with **Plan-space** search
 - Partial-order planning
- Planning with propositional logic
- Analysis of planning approaches



2

What is Planning



- Generate **sequences of actions** to perform tasks and achieve objectives.
 - ☐ States, actions and goals
- **Search** for solution over abstract space of plans.
- Classical planning environment: fully observable, deterministic, finite, static and discrete.
- Assists humans in practical applications
 - ☐ design and manufacturing
 - ☐ military operations
 - ☐ games
 - ☐ space exploration



3

Difficulty of real world problems



- Assume a problem-solving agent using some search method ...
 - ☐ Which actions are relevant?
 - ☐ Exhaustive search vs. backward search
 - ☐ What is a good heuristic functions?
 - ☐ Good estimate of the cost of the state?
 - ☐ Problem-dependent vs, -independent
 - ☐ How to decompose the problem?
 - ☐ Most real-world problems are **nearly** decomposable.



4

Planning language



- What is a good language?
 - **Expressive** enough to describe a wide variety of problems.
 - **Restrictive** enough to allow efficient algorithms to operate on it.
 - Planning algorithm should be able to take advantage of the logical structure of the problem.
 - **Can we use PL or FOL???**
- STRIPS and ADL
 - **ST**anford **R**esearch **I**nstitute **P**roblem **S**olver
 - **A**ction **D**escription **L**anguage

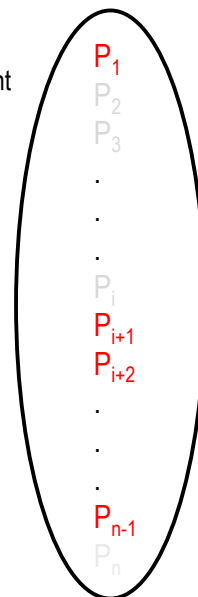


5

General language features



- Representation of **states**
 - Decompose the world in logical conditions and represent a state as a **conjunction of positive ground literals**.
 - Propositional literals: *Poor* $\wedge$ *Unknown*
 - FO-literals (grounded and function-free): *At(Plane1, Melbourne)* $\wedge$ *At(Plane2, Sydney)*
 - Closed world assumption
- Representation of **goals**
 - Partially specified state and represented as a **conjunction of positive ground literals**
 - A goal is **satisfied** in a state if the state contains all literals in goal.



6

General language features



■ Representations of **actions**

- Action = PRECOND + EFFECT

Action(Fly(*p*,*from*, *to*),

PRECOND: $At(p, from) \wedge Plane(p) \wedge City(from) \wedge City(to)$

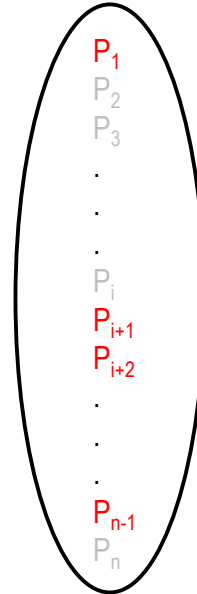
EFFECT: $\neg At(p, from) \wedge At(p, to)$

= action schema (*p*, *from*, *to* need to be instantiated)

- Action name and parameter list
- Precondition (conj. of function-free literals)
- Effect (conj of function-free literals and *P* is True and not *P* is false)

- **Add-list** vs **delete-list** in Effect

□ E.g. $\{(P_3, P_i), \{P_1, P_n\}\}$



7

General language features



■ Representations of **actions**

- Action = PRECOND + EFFECT

Action(Fly(*p*,*from*, *to*),

PRECOND: $At(p, from) \wedge Plane(p) \wedge City(from) \wedge City(to)$

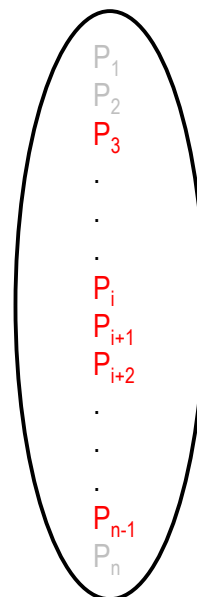
EFFECT: $\neg At(p, from) \wedge At(p, to)$

= action schema (*p*, *from*, *to* need to be instantiated)

- Action name and parameter list
- Precondition (conj. of function-free literals)
- Effect (conj of function-free literals and *P* is True and not *P* is false)

- **Add-list** vs **delete-list** in Effect

□ E.g. $\{(P_3, P_i), \{P_1, P_n\}\}$



8

Language semantics?



■ How do actions affect states?

- An action is applicable in any state that satisfies the precondition.
- For FO action schema applicability involves a substitution θ for the variables in the PRECOND.

$At(P1, Melb) \wedge At(P2, Syd) \wedge Plane(P1) \wedge Plane(P2) \wedge City(Melb) \wedge City(Syd)$

Satisfies : $At(p, from) \wedge Plane(p) \wedge City(from) \wedge City(to)$

With $\theta = \{p/P1, from/Melb, to/Syd\}$

Thus the action is applicable.



9

Language semantics?



■ The result of executing action a in state s is the state s'

- s' is same as s except
 - Any positive literal P in the effect of a is added to s'
 - Any negative literal $\neg P$ is removed from s'

$At(P1, Melb) \wedge At(P2, Syd) \wedge Plane(P1) \wedge Plane(P2) \wedge City(Melb) \wedge City(Syd)$

EFFECT: $\neg At(P1, Melb) \wedge At(P1, Syd)$:

$At(P2, Syd) \wedge Plane(P1) \wedge Plane(P2) \wedge City(Melb) \wedge City(Syd) \wedge$

$At(P1, Syd)$



10

Expressiveness and extensions



- STRIPS is simplified
 - Important limit: function-free literals
 - Allows for propositional representation
 - Function symbols lead to infinitely many states and actions
- Recent extension: Action Description language (ADL)

Action(Fly(p :Plane, from: City, to: City),

PRECOND: $At(p, from) \wedge \neg(from = to)$

EFFECT: $\neg At(p, from) \wedge At(p, to)$

Standardization : *Planning domain definition language (PDDL)*



11

Example: air cargo transport



Init($At(C1, Melb) \wedge At(C2, Syd) \wedge At(P1, Melb) \wedge At(P2, Syd) \wedge Cargo(C1) \wedge Cargo(C2) \wedge$
 $Plane(P1) \wedge Plane(P2) \wedge City(Syd) \wedge City(Melb)$)

Goal($At(C1, Syd) \wedge At(C2, Melb)$)

Action(Load(c, p, a))

PRECOND: $At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge City(a)$

EFFECT: $\neg At(c, a) \wedge In(c, p)$

Action(Unload(c, p, a))

PRECOND: $In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge City(a)$

EFFECT: $At(c, a) \wedge \neg In(c, p)$

Action(Fly($p, from, to$))

PRECOND: $At(p, from) \wedge Plane(p) \wedge City(from) \wedge City(to)$

EFFECT: $\neg At(p, from) \wedge At(p, to)$

[Load(C1,P1,Melb), Fly(P1,Melb,Syd), Load(C2,P2,Syd), Fly(P2,Syd,Melb)]



12

Example: Spare tire problem



Init(*At*(Flat, Axle) $\wedge$ *At*(Spare, trunk))

Goal(*At*(Spare, Axle))

Action(*Remove*(Spare, Trunk))

PRECOND: *At*(Spare, Trunk)

EFFECT: $\neg$ *At*(Spare, Trunk) $\wedge$ *At*(Spare, Ground)

Action(*Remove*(Flat, Axle))

PRECOND: *At*(Flat, Axle)

EFFECT: $\neg$ *At*(Flat, Axle) $\wedge$ *At*(Flat, Ground)

Action(*PutOn*(Spare, Axle))

PRECOND: *At*(Spare, Ground) $\wedge$ $\neg$ *At*(Flat, Axle)

EFFECT: *At*(Spare, Axle) $\wedge$ $\neg$ *At*(Spare, Ground)

Action(*LeaveOvernight*)

PRECOND:

EFFECT: $\neg$ *At*(Spare, Ground) $\wedge$ $\neg$ *At*(Spare, Axle) $\wedge$ $\neg$ *At*(Spare, trunk) $\wedge$ $\neg$ *At*(Flat, Ground) $\wedge$ $\neg$ *At*(Flat, Axle)



This example goes beyond STRIPS: negative literal in pre-condition (ADL description)

13

Example: Blocks world



Init(*On*(A, Table) $\wedge$ *On*(B, Table) $\wedge$ *On*(C, Table) $\wedge$ *Block*(A) $\wedge$ *Block*(B) $\wedge$ *Block*(C) $\wedge$ *Clear*(A) $\wedge$ *Clear*(B) $\wedge$ *Clear*(C))

Goal(*On*(A, B) $\wedge$ *On*(B, C))

Action(*Move*(b, x, y))

PRECOND: *On*(b, x) $\wedge$ *Clear*(b) $\wedge$ *Clear*(y) $\wedge$ *Block*(b) $\wedge$ (b $\neq$ x) $\wedge$ (b $\neq$ y) $\wedge$ (x $\neq$ y)

EFFECT: *On*(b, y) $\wedge$ *Clear*(x) $\wedge$ $\neg$ *On*(b, x) $\wedge$ $\neg$ *Clear*(y)

Action(*MoveToTable*(b, x))

PRECOND: *On*(b, x) $\wedge$ *Clear*(b) $\wedge$ *Block*(b) $\wedge$ (b $\neq$ x)

EFFECT: *On*(b, Table) $\wedge$ *Clear*(x) $\wedge$ $\neg$ *On*(b, x)

Spurious actions are possible: *Move*(B, C, C)



14

Planning with state-space search

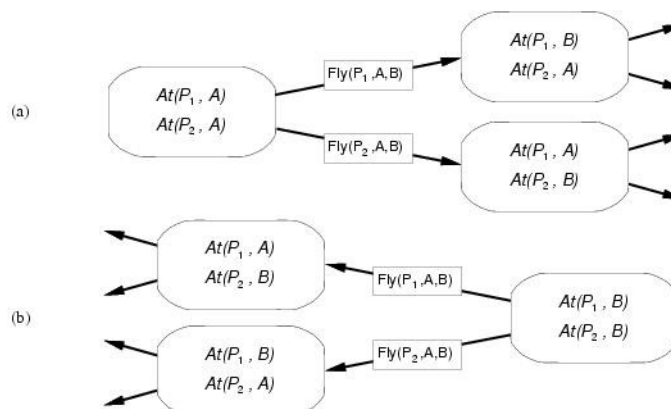


- Both forward and backward search possible
- Progression planners
 - forward state-space search
 - Consider the effect of all possible actions in a given state
- Regression planners
 - backward state-space search
 - To achieve a goal, what must have been true in the previous state.



15

Progression and regression



16

Partial-order planning



- Progression and regression planning are *totally ordered plan search* forms.
 - They cannot take advantage of problem decomposition.
 - Decisions must be made on how to sequence actions on all the subproblems
- Least commitment strategy:
 - Delay choice during search



17

Shoe example



Goal(RightShoeOn $\wedge$ LeftShoeOn)

Init()

Action(RightShoe, PRECOND: RightSockOn
 EFFECT: RightShoeOn)

Action(RightSock, PRECOND:
 EFFECT: RightSockOn)

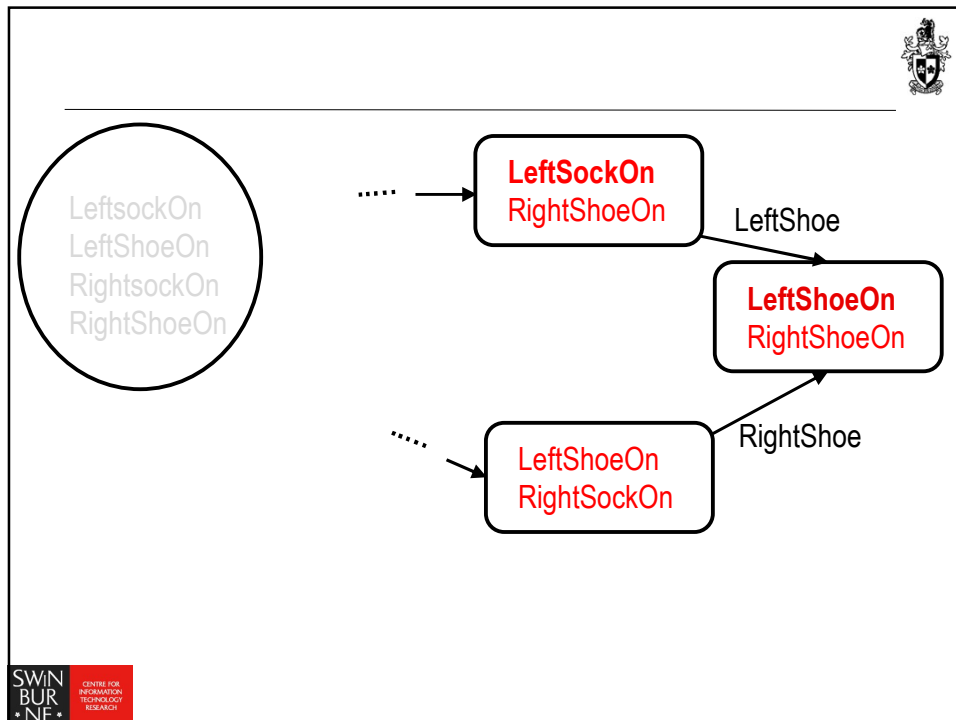
Action(LeftShoe, PRECOND: LeftSockOn
 EFFECT: LeftShoeOn)

Action(LeftSock, PRECOND:
 EFFECT: LeftSockOn)

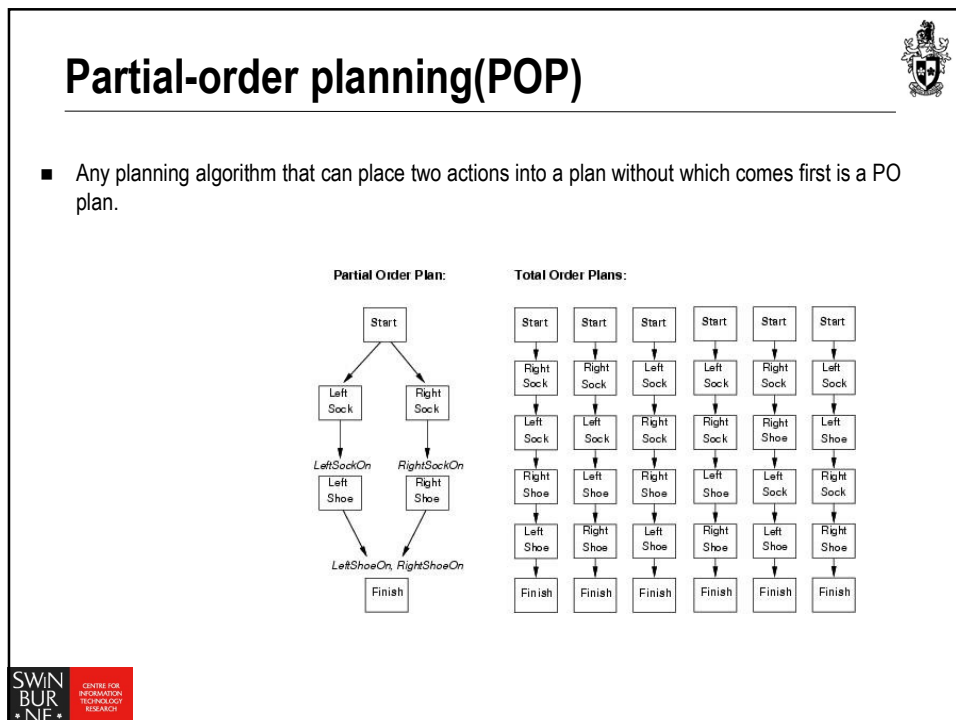
Planner: combine two action sequences (1)leftsock, leftshoe
 (2)rightsock, rightshoe



18



19



20

POP as a search problem



- States are (mostly unfinished) plans.
 - The empty plan contains only start and finish actions.
- Each plan has 4 components:
 - A set of actions (steps of the plan) ~~$A < B, B < C, C < A$~~
 - A set of ordering constraints: $A < B$ (A before B)
 - **Cycles** represent contradictions.
 - A set of causal links $A \xrightarrow{p} B$
 - The plan may not be extended by adding a new action C that **conflicts** with the causal link. (if the effect of C is $\neg p$ and if C could come after A and before B)
 - A set of **open preconditions**.
 - If precondition is not achieved by action in the plan.



21

Example of final plan



- Actions={Rightsock, Rightshoe, Leftsock, Leftshoe, Start, Finish}
- Orderings={Rightsock < Rightshoe; Leftsock < Leftshoe}
- Links={**Rightsock**->**Rightsockon** -> **Rightshoe**, **Leftsock**->**Leftsockon**-> **Leftshoe**, **Rightshoe**->**Rightshoeon**->**Finish**, ...}
- Open preconditions={}



22

POP as a search problem



- A plan is *consistent* iff there are no cycles in the ordering constraints and no conflicts with the causal links.
- A consistent plan with no open preconditions is a *solution*.
- A partial order plan is executed by repeatedly choosing *any* of the possible next actions.
 - This flexibility is a benefit in non-cooperative environments.



23

Solving POP



- Assume propositional planning problems:
 - The initial plan contains *Start* and *Finish*, the ordering constraint $Start < Finish$, no causal links, all the preconditions in *Finish* are open.
 - Successor function :
 - picks one open precondition p on an action B and
 - generates a successor plan for every possible consistent way of choosing action A that achieves p .
 - Test goal



24

Enforcing consistency



- When generating successor plan:
 - ☐ The causal link $A \rightarrow p \rightarrow B$ and the ordering constraint $A < B$ is added to the plan.
 - ☐ If A is new also add start $< A$ and $A < B$ to the plan
 - ☐ Resolve conflicts between new causal link and all existing actions
 - ☐ Resolve conflicts between action A (if new) and all existing causal links.



25

Process summary



- Operators on partial plans
 - ☐ Add link from existing plan to open precondition.
 - ☐ Add a step to fulfill an open condition.
 - ☐ Order one step w.r.t another to remove possible conflicts
- Gradually move from incomplete/vague plans to complete/correct plans
- Backtrack if an open condition is unachievable or if a conflict is irresolvable.



26

Example: Spare tire problem



Init($At(Flat, Axle) \wedge At(Spare, trunk)$)

Goal($At(Spare, Axle)$)

Action(*Remove*(*Spare*, *Trunk*))

PRECOND: $At(Spare, Trunk)$

EFFECT: $\neg At(Spare, Trunk) \wedge At(Spare, Ground)$

Action(*Remove*(*Flat*, *Axle*))

PRECOND: $At(Flat, Axle)$

EFFECT: $\neg At(Flat, Axle) \wedge At(Flat, Ground)$

Action(*PutOn*(*Spare*, *Axle*))

PRECOND: $At(Spare, Ground) \wedge \neg At(Flat, Axle)$

EFFECT: $At(Spare, Axle) \wedge \neg At(Spare, Ground)$

Action(*LeaveOvernight*)

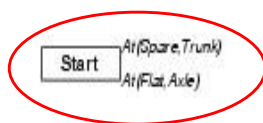
PRECOND:

EFFECT: $\neg At(Spare, Ground) \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, trunk) \wedge \neg At(Flat, Ground) \wedge \neg At(Flat, Axle)$



27

Solving the problem

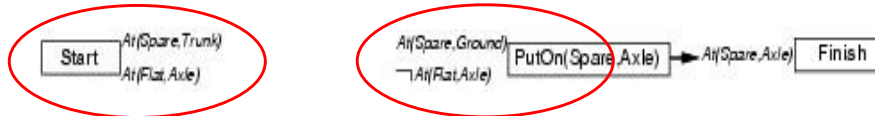


- Initial plan: Start with EFFECTS and Finish with PRECOND.



28

Solving the problem

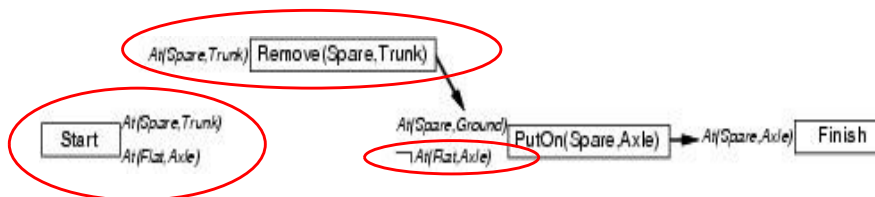


- Initial plan: Start with EFFECTS and Finish with PRECOND.
- Pick an open precondition: $At(Spare, Axle)$
- Only $PutOn(Spare, Axle)$ is applicable
- Add causal link: $PutOn(Spare, Axle) \xrightarrow{At(Spare, Axle)} Finish$
- Add constraint : $PutOn(Spare, Axle) < Finish$



29

Solving the problem

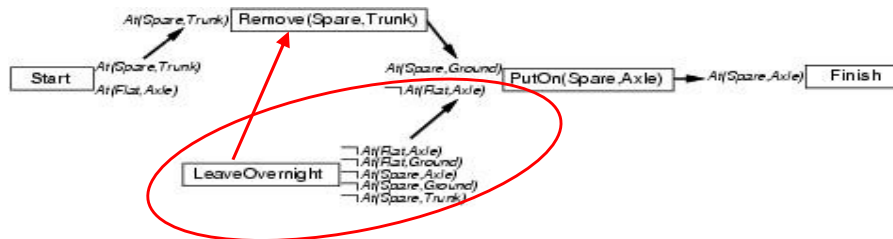


- Pick an open precondition: $At(Spare, Ground)$
- Only $Remove(Spare, Trunk)$ is applicable
- Add causal link: $Remove(Spare, Trunk) \xrightarrow{At(Spare, Ground)} PutOn(Spare, Axle)$
- Add constraint : $Remove(Spare, Trunk) < PutOn(Spare, Axle)$



30

Solving the problem

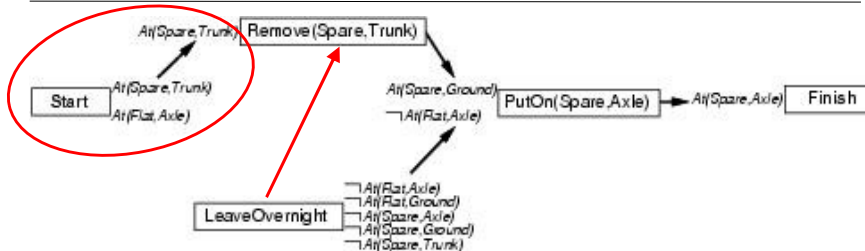


- Pick an open precondition: $\neg At(Flat, Axle)$
- *LeaveOverNight* is applicable
- conflict: *LeaveOverNight* also has the effect $\neg At(Spare, Ground)$
- $Remove(Spare, Trunk) \xrightarrow{At(Spare, Ground)} PutOn(Spare, Axle)$
- To resolve, add constraint : $LeaveOverNight < Remove(Spare, Trunk)$



31

Solving the problem

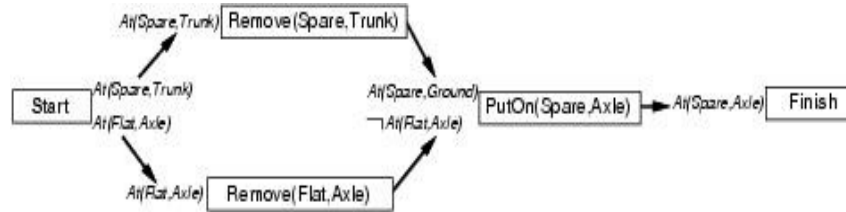


- Pick an open precondition: $At(Spare, Trunk)$
- Only *Start* is applicable
- Add causal link: $Start \xrightarrow{At(Spare, Trunk)} Remove(Spare, Trunk)$
- Conflict: of causal link with effect $At(Spare, Trunk)$ in *LeaveOverNight*
 - No re-ordering solution possible.
- backtrack



32

Solving the problem



- Remove *LeaveOverNight* and causal links
- Add *RemoveFlatAxle* and complete the plan



33

Planning with propositional logic



- Planning can be done by proving theorem in situation calculus.
- Here: test the *satisfiability* of a logical sentence:

initial state $\wedge$ *all possible action descriptions* $\wedge$ *goal*

- Sentence contains propositions for every action occurrence.
 - ☐ A model will assign true to the actions that are part of the correct plan and false to the others
 - ☐ An assignment that corresponds to an incorrect plan will not be a model because of inconsistency with the assertion that the goal is true.
 - ☐ If the planning is unsolvable the sentence will be unsatisfiable.



34

Analysis of planning approach



- Planning is an area of great interest within AI
 - Search for solution
 - Constructively prove a existence of solution
- Biggest problem is the combinatorial explosion in states.
- Efficient methods are under research
 - E.g. divide-and-conquer

